As a Playwright + JS automation expert, here's how I approach automating tables with duplicate/identical rows.

Key Challenges with Duplicate Rows

- Simple `nth-child()` or index-based locators become unreliable.
- You need **stable, data-driven identification** using multiple columns or unique combinations.
- The table on https://selectorshub.com/xpath-practice-page/ (under **iframe and table** section) has many near-identical rows (e.g., repeated "windows + chrome + Samsun + India").

Recommended Strategies (in order of preference)

1. Best: Use `locator.filter()` with multiple conditions (Playwright-native, most reliable)

```
JavaScript

const { test, expect } = require('@playwright/test');

test('Handle table with duplicate rows inside iframe', async ({ page }) =>
{
  await page.goto('https://selectorshub.com/xpath-practice-page/');

  // 1. Switch to the iframe containing the table
  const iframe = page.frameLocator('iframe'); // Adjust selector if needed
(use SelectorsHub for precise one)
  // Or more specific: page.frameLocator('iframe[src*="..."]') or by
title/name

  const table = iframe.locator('table'); // or '#tablepress-1' etc.

  // Example: Find rows where OS=windows, Browser=chrome, City=Samsun,
Country=India
  const targetRows = table.locator('tbody tr').filter({
    has: iframe.locator('td', { hasText: 'windows' }),   // Column 1 (OS)
  }).filter({
    has: iframe.locator('td', { hasText: 'chrome' }),    // Column 2
(Browser)
```

```
  }).filter({
    has: iframe.locator('td', { hasText: 'Samsun' }),    // Column 3
(City)
  }).filter({
    has: iframe.locator('td', { hasText: 'India' }),     // Column 4
(Country)
  });

  const count = await targetRows.count();
  console.log(`Found ${count} matching rows`);

  // Interact with them (e.g., click checkbox in first column, or any
action)
  for (let i = 0; i < count; i++) {
    const row = targetRows.nth(i);

    // Example actions
    await row.locator('input[type="checkbox"]').check(); // if present
    const employeeName = await row.locator('td:nth-
child(4)').textContent(); // adjust column

    console.log(`Processed row ${i+1}: ${employeeName}`);
  }

  await expect(targetRows).toHaveCount(10); // example assertion (adjust)
});
```

2. XPath with multiple conditions (very powerful for complex matching)

```
JavaScript

const matchingRows = iframe.locator(`//tr[
  td[1][contains(text(),'windows')] and
  td[2][contains(text(),'chrome')] and
  td[3][contains(text(),'Samsun')] and
  td[4][contains(text(),'India')]
]`);

await expect(matchingRows).toHaveCountGreaterThan(5);
```

3. Extract entire table as data structure (great for validation/reporting)

```
JavaScript

async function getTableData(tableLocator) {
  const rows = await tableLocator.locator('tbody tr').all();
  const data = [];

  for (const row of rows) {
    const cells = await row.locator('td').allTextContents();
    data.push({
      os: cells[0]?.trim(),
      browser: cells[1]?.trim(),
      city: cells[2]?.trim(),
      country: cells[3]?.trim(),
      // add more columns as needed
    });
  }
  return data;
}

// Usage
const tableData = await getTableData(table);
const indiaChromeRows = tableData.filter(row =>
  row.os === 'windows' &&
  row.browser === 'chrome' &&
  row.city === 'Samsun' &&
  row.country === 'India'
);

console.log('Matching rows:', indiaChromeRows.length);
```

4. Using `:has()` + `:has-text()` (clean CSS-style)

```
JavaScript

const targetRow = table.locator('tr').filter({
  hasText: 'windows'
}).filter({
  has: page.locator('td:has-text("chrome")')
  // chain more filters
});
```

Handling Iframe Best Practices

1. **Always use** `frameLocator` — never mix page + frame selectors incorrectly.
2. Prefer **SelectorsHub browser extension** (it auto-generates perfect Playwright locators for iframes, nested iframes, shadow DOM, and tables).
3. Wait strategies:

```
JavaScript
await iframe.locator('table').waitFor({ state: 'visible' });
```

General Table Automation Tips (Playwright JS)

- Prefer **text/content-based filtering** over fragile indexes.
- For actions in a row → scope the locator: `row.locator('button.delete')`.
- Handle pagination if present: loop + `next` button until no more pages.
- Performance: Use `all()` sparingly on very large tables; prefer server-side filtering when possible.
- Assertions: `toHaveCount()`, `toContainText()`, custom data validation.

Full Example Script Structure

```
JavaScript
import { test, expect } from '@playwright/test';

test.describe('Table with Duplicates', () => {
  test('Validate and interact with duplicate rows', async ({ page }) => {
    await page.goto('https://selectorshub.com/xpath-practice-page/');

    const frame = page.frameLocator('your-iframe-selector'); // get
precise one via dev tools
    const table = frame.locator('table#tablepress-1'); // example id

    // Your scenario-specific logic here...
  });
});
```

Pro Tip: Install SelectorsHub extension → inspect any cell in the table → it will give you the

Pro Tip: Install **SelectorsHub** extension → Inspect any cell in the table → It will give you the exact Playwright locator (including iframe handling). This saves huge time on complex pages.

Would you like me to:

- Write the complete ready-to-run test for a specific scenario on that page?
- Handle checkboxes/actions in those duplicate rows?
- Add data-driven approach with test fixtures?
- Handle nested iframes or shadow DOM + table combo?

Just give more details on the exact user flow (e.g., "find all Samsun+India rows and click edit").